Name : T. kaushiko
Roll no: 240701239

S.NO	DATE	INDEX	MARKS	SIGN
1.	20/7/26	Study of various Network Commands using Linux and Windows.	—	10/8
2.	28/7/26	Study of Network Cables.	—	10/8
3	3/8/26	Experiments on CISCO packet Tracker.	—	11/8
4	10/8/26	Setting up a local Area network		
5	11/8/26	Experiments on packet capture tool		
6.	17/8/26	Hamming code		06/10/26
7	24/8/26	Sliding window protocol		
8.	28/9/26	N-map Live Host Discovery		
9	29/9/26	Subnetting in cisco packet tracer simulator		
10	05.10.26	Internetworking with routers in cisco packet tracker		

S-NO	Date	Index	Sign
11	06.10.2026	Simulate RIP using cisco packet tracer.	
12	12.10.2026	End to End communication at Transport Layer	
13	13.10.2026	Implement your own ping program	
14.	13.10.2026	Write a code using RAW packet to implement packet sniffing	

Completed
13/10/26

20-07-2026

Experiment-1

Study of Various Network Commands Used in Linux and Windows

Aim :-

Study and Execute the Various Network Commands Using (Linux) fedora

Basic Networking Commands :

arp -a (fedora)

ARP is short form of address resolution protocol

It will show the IP address of your computer along with IP address and MAC address of your router

-gateway (10.0.2.2) at 52:55:0a:00:02:02

hostname (fedora)

This is the simplest of all TCP/IP commands. It Simply displays the name of your computer.

fedora

IPConfig /all

This command display detailed configuration information about your TCP/IP connection including Router, Gateway DNS, DHCP and type of Ethernet adapter in your System.

hostname	DESKTOP-NC3QMNB
Physical Address	F8-ED-FC-5B-DA-A8
DHCP Enabled	No
IPv4 Address	172.16.10.145
Subnet Mask	255.255.252.0

nbtstat -a (cd)

This command helps solve problems with NetBIOS name resolution (Nbt stands for NetBIOS over TCP/IP)

NBTSTAT [[-a RemoteName] [-A IP address] [-c] [-n] [-r] [-R] [-RR] [-s] [-S] [interval]]

-a (adapter status) -c (cache) -r (resolved)
-A (Adapter status) -n (names) -R (Reload)
-S (sessions)
-s (sessions)

eg NBTSTAT -c
Ethernet 2:
Node IPAddress: [192.168.56.1] Scope Id: []
no name in cache.

netstat (fedora)
It displays a variety of statistics about computers active TCP/IP connections.

Proto	Refcnt	Flags	Type	State	I-Node	Path
Unix	3	[ ]	STREAM	CONNECTED	25395	/run/user/1000/bus
Unix	3	[ ]	STREAM	CONNECTED	26159	

nslookup (nslookup www.google.com) (cd)
It is a tool used to perform DNS lookups in Linux. It's display DNS details such as IP address of particular computer, Mx records for domain.

Server: Unknown
Address: 172.16.8.1
Name: www.google.com
Address: 2001:4860:4827:7100::
142.251.156.119
142.251.157.119

Pathping (cd)
Pathping is unique to Windows and basically combination of the ping and Tracert commands.

Pathping [-g host-list] [-h maximum_hops] [-i address] [-n] [-p period] [-q num-queries] [-w timeout] [-4] [-6]

ping (Ping www.google.com)

(Packet INternet Groper) Command is the best way to test Connectivity between two nodes. (cd)

① Pinging www.google.com [142.251.152.119] with 32 bytes of data

Reply from 142.251.152.119 : bytes = 32 time = 4ms TTL = 119

:

Ping statistics for 142.251.152.119

Packets: sent = 4, Received = 4, lost = 0 (0% loss)

② Ping 172.16.10.144 (cd)

Pinging 172.16.10.144 with 32 bytes of data.

Request timed out.

:

Ping statistics for 172.16.10.144

Packets: Sent = 4, Received = 0, Lost = 4 (100% loss)

③ Ping (localhost) fedora (fedora)

Pinging ~~DESK~~ fedora (fe80::68e6:66f:2f87:bc1b%enpos3) 56 data bytes

64 bytes from fedora (fe80::68e6:66f:2f87:bc1 6%enpos3):

icmp-seq=1 ttl=64 time=0.081 ms

route (fedora)

route command is used to show the IP routing table.

Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
default	_gateway	0.0.0.0	UG	100	0	0	enpos3
10.0.2.0	0.0.0.0	255.255.255.0	U	100	0	0	enpos3.

Some important Linux networking commands

1. ip

The ip command is one of the Basic command every administrator will need in daily work, from setting up new systems and assigning IPS to troubleshooting existing system.

ip <OPTIONS> <OBJECT> <COMMAND>

Here are some common use case for the ip command

① To show the IP addresses assigned to an interface on your server.

To Enter inside sudo
sudo -i

ip address show

1: lo: <LOOPBACK, UP, LOWERUP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo

② To assign an IP to an interface, for example enps03

ip address add 192.168.1.254/24 dev ~~enps03~~ enps03

after this command enter ip address show (or) ip address show enps03

altname enx0800270 2dfb8
inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enps03
valid_lft 83626 sec scope global ~~enps03~~ 83626 sec
inet 192.168.1.254/24 scope global enps03

③ To delete an IP on an interface

ip address del 192.168.1.254/24 dev enps03

ip address show

inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enps03
valid_lft 83626 sec preferred_lft 83569 sec
inet6 fd17:625c:f037:2:4994:1662:2608:52d7/64
scope global dynamic noprefixroute

④ ip link set enps03 up

~~ip address show~~ ip link show enps03

enps03: <BROADCAST, MULTICAST, UP, LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000

link/ether 08:00:27:02:df:8e brd ff:ff:ff:ff:ff:ff
altname enx08002702df8e

⑤ ip link set enp0s3 down
ip link show enp0s3

enp0s3: <BROADCAST, MULTICAST> mtu 1500 qdisc fq_codel DOWN mode DEFAULT group default qlen 100
link/ether 08:00:27:02:df:8e brd ff:ff:ff:ff:ff:ff

⑥ ip link set enp0s3 promisc on
ip link show enp0s3

enp0s3: <BROADCAST, MULTICAST, PROMISC> mtu 1500 adisc fq_codel state DOWN DEFAULT group default qlen 1000
link/ether 08:00:27:02:df:8e brd ff:ff:ff:ff:ff:ff

⑦ ip route add ~~feature~~ default via 192.168.1.254 dev enp0s3

⑧ ip route get
10.10.1.14 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 1000

2. ifconfig

The ifconfig command was/is a staple in many sysadmin's tool belt for configuring and troubleshooting network

ifconfig

enp250: flag=4163<UP, BROADCAST, RUNNING, MULTICAST> mtu 1500
inet 172.16.9.200 netmask 255.255.252.0 broadcast 172.16.11.255
:
lo: flags=73<UP, LOOPBACK, RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0

3. mtr

MTR (Matt's traceroute is a program with a command-line interface that serves as a network diagonostic and troubleshooting tool.

mtr <options> hostname/IP

i) mtr

	Packets			Pings			
Host	Loss%	Snt	Last	Avg	Best	Wrst	StDev
1. localhost	0.0%	93	0.1	0.1	0.1	0.1	0.1

ii) mtr google.com

	Packets			Pings			
Host	Loss%	Snt	last	Avg	Best	Wrst	stDev
1. _gateway	1.7	61	0.3	0.3	0.3	14.2	1.8
2. 137.97.163.129	1.0%	98	1.5	2.9	1.2	13.4	2.1

iii) mtr -b google.com

	Packet			Pings			
Host	Loss%	Snt	last	Avg	Best	Wrst	stDev
1. _gateway	18.9	299	0.4	0.3	0.2	9.2	0.1
2. 137.97.163.129	1.3	311	1.5	2.4	1.1	16.4	2.1

iv) mtr -c 10

	Packet			Pings		
Host	Loss%	Snt	last	Avg	Best	Wrst	Stdev
1. localhost	0.0	5	0.1	0.1	0.1	0.1	0.0

4. tcpdump

The tcpdump command is designed for capturing and displaying packets

① dnf install -y tcpdump

updating and loading reesposi tories

Fedora 43 - X86_64 - updates 100% | 17.0 KiB/s | 64 KiB | 00m00s

② sudo tcpdump -D

enp0s3 [up, Running, Connected]

any (Pseudo-device that capture on all interfaces) [up, Running]

lo [up, Running, Loopback]

usbmon2 (Raw USB traffic, bus number 2]

③ sudo tcpdump -i enp0s3

dropped privs to tcpdump

listening on enp0s3, link-type EN10MB (Ethernet)

08:32:42.187998 IP fedora.44536 > time.cloudflare.com.ntp: NTPv4, client, length 48

08:32:42.188333 IP time.cloudflare.com.ntp > fedora.44536: NTPv4, server, length 48.

④ tcpdump -i ~~eth0~~ enp0s3 -c 10

dropped privs to tcpdump

listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes.

08:38:10.724126 IP fedora.48384 > ec2-3-111-45-100.ap-south-1.compute.amazonaws.com.ntp: NTPv4, client, length 48

⑤ tcpdump -i enp0s3 -C 10 host 8.8.8.8

listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes

08:38:30.135661 IP fedora.52412 > 172.16.8.1.domain: 34643+AAAA? fedoraproject.org. (35)

⑥ tcpdump -i eth0 src host 8.8.8.8

tcpdump: verbose output suppressed, use -v[v] for full protocol decoder

listening on eth0, link-type ENIONB (Ethernet), snapshot length 262144 length

13:33:07.529734 IP dns.google > 10.0.2.15 : ICMP echo reply, id 33107, seq 46, length 64.

⑦ tcpdump -i eth0 dst host 8.8.8.8

13:35:01.231291 IP 10.0.2.15 > dns.google: ICMP echo request, id 33107, seq 157, length 64

14 packets captured
14 packets received by filter
0 packet dropped by kernel

⑧ tcpdump -i eth0 net 10.0.2.0 mask 255.255.255.0

(or)

tcpdump -i eth0 net 10.0.2.0/24

13:46:43.280056 IP 10.0.2.15.39581 > 10.39.4.180.domain. 19776+ A? 951.gstatic.com (33)

35 packets captured
35 packet received by filter
0 packet dropped by kernel.

⑨ tcpdump -i eth0 port 53

18:50:05.853192 IP 10.0.2.15.34956 > 10.39.4.180.domain 59557 A? prod-dynamite-prod-04-us-signaler-pa.goog.6.com (76)

8 packet captured
9 packet received by filter
0 packet dropped by kernel → ping www.google.com

⑩ tcpdump -i eth0 -c 10 host www.google.com and port 443

13:56:42.648546 IP 10.0.2.15.46676 > 142.251.158.119.https: Flags [S], seq 2826149958, win 64240, options [mss 1460, sackOK, TS Val 2112807762 ecr 0, nop, wscale 8], length 0

packet captured
packet received by filter

(11) etho not port 53 and not port 25

14:04:48.886015 IP 10.0.2.15.57480 > lcmaa-ar-in-f14.1e100.net.https:
Flags [P.], seq 572566992:572565031, ack 2199490,
win 60511, length 39.

⋮

16 packet captured
16 packet received by filter

5. ping

ping is a tool that verifies IP-level connectivity to another TCP/IP computer by sending Internet control Message protocol ICMP Echo Request message.

① ping google.com

PING google.com (142.250.67.46) 56(84) bytes of data

64 bytes from pnmaa-bb-in-f14.1e100.net (142.250.67.46): icmp_seq=1 ttl=112 time=27.7 ms

⋮

— google.com.ping statistics —

20 packets transmitted, 20 received 0% packet loss, time 19213ms.

② ping -c 10 google.com.

64 bytes from pnmaa-bb-in-f14.1e100.net (142.250.67.46): icmp-seq=1 ttl=112 time=19.2 ms

⋮

— google.com.ping. statistics —

10 packet transmitted, 10 recieved, 0% packet loss, time 9023ms

Configuring an Ethernet Connection by Using nmcli

If you connect a host to the network over Ethernet, you can manage the connection's setting on the command

① nmcli connection show

NAME	UUID	TYPE	DEVICE
Wired Connection 1	d7d7365f-c2d9-3cb3-ab4f-5d63bd7fe7e9	ethernet	eth0
lo	34fob00e-983e-4941-a0ab-bcdesc7ff938	loopback	lo

② nmcli connection add type ethernet ifname eth0 con-name "Wired Connection 1"

Connection 'Wired Connection 1' (a00c990-7513-4c63-92ee-40edd07c4adc) successfully added.

nmcli connection show

NAME	UUID	TYPE	DEVICE
Wired connection 1	d7d7365f-c2d9-3cb3-ab4f-	ethernet	eth0
lo	34fob00e-983e-4941-a0ab-bcdesc7	loopback	lo
wired connection 1	a00c990-7513-4c63-92ee-40edd07c4adc	ethernet	-

③ nmcli connection up "Wired Connection 1"

Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/3)

④ ip address show eth0

eth0: <BROADCAST, MULTICAST, UP, LOWER_UP> mtu 1500 qdisc fq codel state UP group default qlen 1000

link/ether 08:00:27:8a:35:de brd ff:ff:ff:ff:ff:ff

inet 192.0.2.1/24 brd 192.0.2.255 scope global noprefixroute eth0

⑤ ip route show

default via 192.0.2.254 dev eth0 proto static metric 100

192.0.2.0/24 dev eth0 proto kernel scope link src 192.0.2.1 metric 100

⑥ ip -6 route show

fe80::/64 dev eth0 proto kernel metric 1024 pref medium

⑦ Ping kali

PING kali (127.0.1.1) 56(84) bytes of data.

64 bytes from kali (127.0.1.1) : icmp-seq =1 ttl=64 time=0.146 ms

⋮

18 packets transmitted, 18 received, 0% packet loss, time 17400ms

rtt min/avg/max/mdev = 0.038/0.059/0.146/0.22 ms.

Student observation :-

1. Which command is used to find the reachability of a host machine from your device.

The Ping command is used to check whether a host machine is reachable from your device.

2. Which command gives the details of hops taken by a packet to reach its destination?

The mtr command shows the route form a computer to a specified host and provides information about each hop

3. Which command displays the IP Configuration of your machine?

ip address show

This command display the IP addresses assigned to the network interfaces

4. Which Command displays the TCP port status on your machine?

netstat.

This command displays information about active TCP/IP connections and network statistics.

5. Write the command to modify the IP configuration in a Linux machine.

To assign an IP address

ip addres add 192.168.1.254/24 dev enp0s3

To delete an IP address

ip address del 192.168.1.254/24 dev enp0s3

EXPERIMENT-2

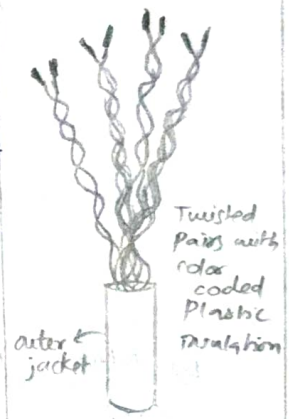
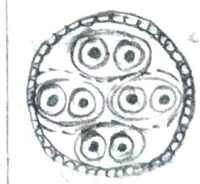
STUDY OF NETWORK CABLES

Aim :- To study of different types of Network cables.

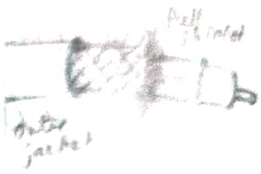
a) Understand different types of network cable.

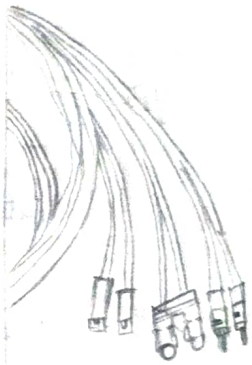
Different type of cables used in networking are

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) Cable
3. Coaxial Cable
4. Fibre Optic Cable.

Cable type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application / Use	Image
UTP	Category 3	10 bps	Adv • cheaper in cost • Easy to install as they have a smaller over diameter disadv • More prone to (EMI) Electromagnetic interference and noise	10 Base-T Ethernet Fast Ethernet Gigabit Ethernet Fast Ethernet Gigabit Ethernet	
	Category 5	Up to 100 Mbps			
	Category 5e	1 Gbps			
STP	Category 6, 6a	10 Gbps	Adv • Shielded • Faster than UTP • Less susceptible to noise and interference disadvantage • Expensive • greater installation effort	Gigabit Ethernet 10G Ethernet (SSM) widely used in data Centre Gigabit Ethernet, 10G Ethernet (100m)	 
SSTP	Category 7	10 Gbps			

Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	adv • High bandwidth • Immune to interference • Low loss bandwidth • Versatile disadv • Limited distance • Cost • Size is bulky	Speed of signal is from Television network high speed Internet connection
fibre optics cable	Single mode Multi mode	100 Gbps	Adv • High speed • High bandwidth • High security • Long distance disadv • Expensive • Requires skilled installers	Maximum distance of fibre optics cable is around 100 meters.

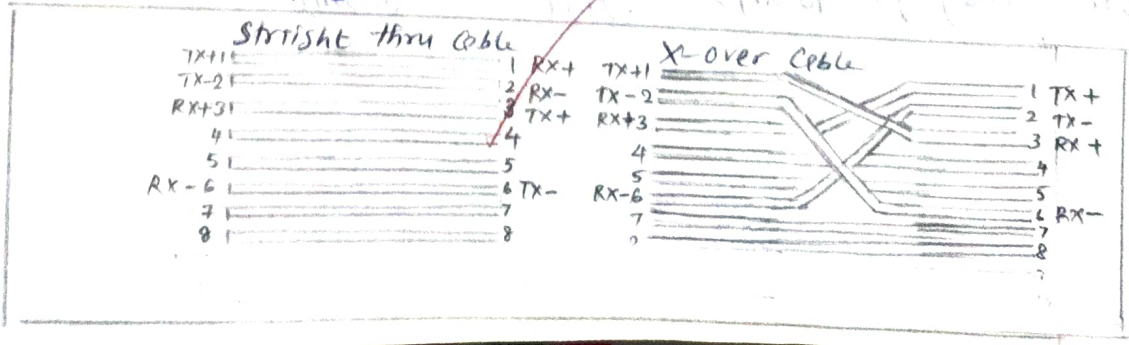


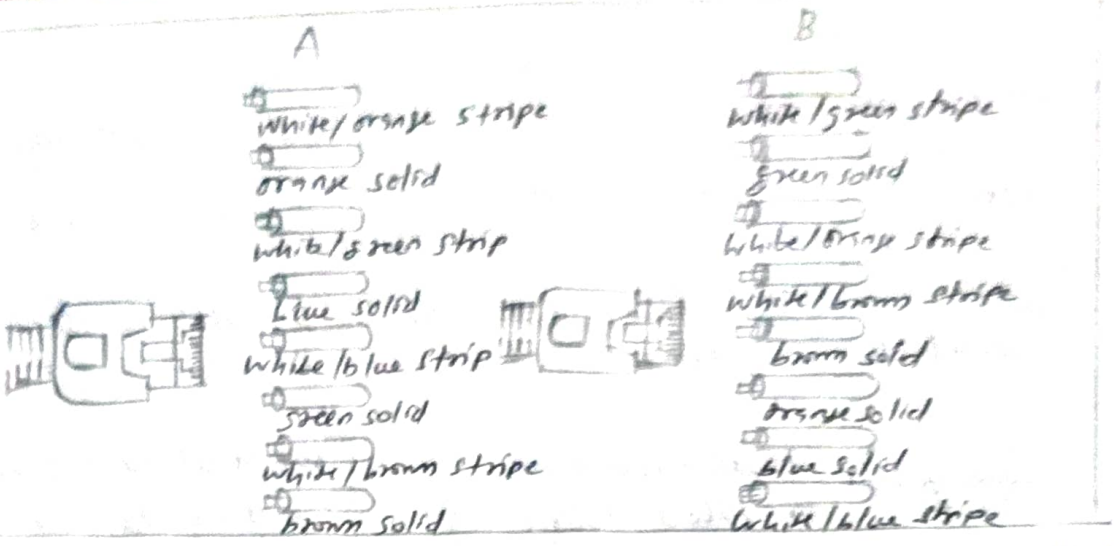


b) Make Your Own Ethernet Cross-Over Cable / Straight Cable

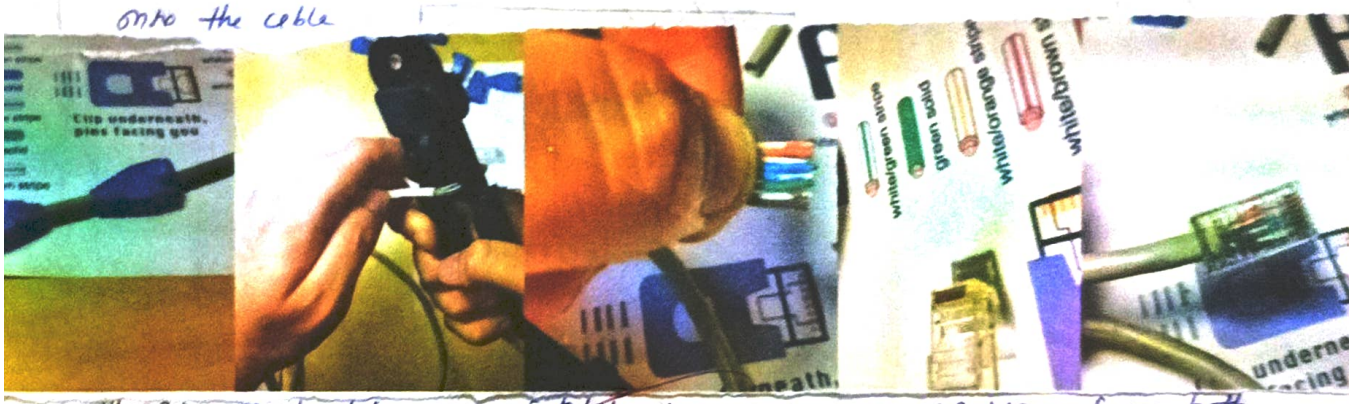
Tool and parts needed:

- Ethernet Cabling. CAT5e is certified for Gigabit Support but CAT5 Cabling works well, just over shorter distances.
- A crimping tool. This is an-in-one networking tool shaped to push down the pins in the plug and strip and cut the Shielding off the cables
- Two RJ45 plugs
- Optional two plug shields





Step 1:- To start construction of the device, begin by threading shields onto the cable



Step2:- Next, strip approximately 1.5 cm of cable shielding from both ends. The crimping tool has a round area to complete this task

Step3:- After, you will need to untangle the wires, there should be four "twisted pairs" Refering back to the sheet, arrange them from top to bottom. One end should be an arrangement A and other in B

Step4:- once the order is correct, bunch them together in a line, and if there are any that stick out farther then others, snip them back to create an even level. The difficult aspect is placing these into the RJ45 plug without messing up the order. To do so, hold the plug with the clip side facing away from you and have the gold pins. facing toward you, as shown.

Step5:- Next, push the cable right in, The notch at the end of the plug needs to be just over the cable shielding. and if it isn't that means that you stripped off too much shielding. simply snip the cables back a little more

Step6:- After the wires are securely sitting inside the plug. Insert it into the crimping tool and push down

Step7:- Lastly, repeat for the other end using diagram B (to make a crossover cables) using diagram A (to make a straight through cable)

Student Observation :-

1. What is the difference between cross cable and straight cable ?

feature	Straight Through cable	Cross Over Cable
Wiring Standard	Same Standard at both ends	Different standards at each end.
Pin arrangement	T568A - T568A (or) T568B - T568B	T568A at one end and T568B at the other
Example	PC to switch / Router	PC to PC
Data transmision	Transmit and receive Pairs remain in the same position	Transmit and receive Pairs are crossed.

2. Which table is used to connect two pcs?
Crossover cable is traditionally used to connect two Pcs directly.

3. Which Cable is used to connect a Router/switch to a PC?
A Straight - Through Cable is used to connect a PC to a Switch or Route

4. Category of Twisted - Pair Cable Used in the Lab
The twisted - pair cable commonly used in computer networking laboratories is :
Category 5e (cat5e) UTP Cable

Property	Details
Cable type	Unshielded Twisted Pair (UTP)
Category	cat5e
Number of pairs	4 pairs
Number of wires	8 wires
Maximum speed	Up to 1Gbps
Common Use	Connecting PCs, switches, routers, and network sockets.

5. Write down your understanding, challenges faced and output received while making a twisted pair cross / straight cable.

Understanding : I learned how to arrange to twisted pair wires using the T568A and T568B standards and make straight-through and crossover cables.

Challenges : the main difficulty was arranging the wires in the correct color order and inserting them properly into the RJ45 connector.

output : The cable was successfully crimped and tested using a cable tester, and proper connectivity was obtained.

3. Experiments on CISCO PACKET TRACKER (Simulation Tool):

Aim: To study the Packet tracer tool Installation and User Interface Overview.

c) To Understand environment of CISCO PACKET TRACER to design simple network

INTRODUCTION:-

A Simulator, as the name suggests, simulates network devices and its environment. Packet Tracker is an exciting network design, simulation and modelling tool.

1. It allow you to model complex systems without the need for dedicated equipment.
2. It's available for both the Linux and Windows desktop environments.
3. Protocols in Packet Tracer are coded to work and behave in the same way as they would on real hardware.

INSTALLING PACKET TRACER

To download packet Tracer, go to https://www.netced.com and log in with your cisco Networking Academy credentials; then, click on the Packet Tracker graphic and download the Package appropriate for your Operating System.

Window:-

Installation in window is pretty simple and straightforward; the setup comes in a single file named PacketTracer_setup 6.0.1.exe. Open this file to begin the setup wizard, accept the license agreement, choose a location, and start the installation

USER INTERFACE OVERVIEW:-

The layout of Packet tracer is divided into several components. The Components of the Packet Tracer interface are as follows: match the numbering with explanations.

1. Menu bar - This is a common menu found on all software applications; it is used to open, save, print change preferences, and so on
2. Main toolbar - this bar provides shortcut icons to menu options that are commonly accessed, such as open, save, zoom, undo, and redo, and on the right hand side is an icon for entering network information
3. Logical / physical workspace tabs - These tabs allow you to toggle b/w the logical and physical work areas.
4. ~~Common tools bar~~ - Workspace - This is the area where topologies are created and simulations are displayed.
5. Common tools bar - This is the toolbar provides controls for manipulating topologies, such as select, move layout, place note, delete, inspect, resize shape, and add simple / complex PDU
6. Real-time / Simulation tabs - These tabs are used to toggle between the real and simulation modes. Buttons are also provided to control the time and to capture the Packets.
7. Network component box - This component contains all of the network and end devices available with packet Tracer, and is further divided into two areas: Area 7a: Device-type Selection box - This area contains device categories Area 7b: Device-Specific selection box - When a device category is selected, this selection box displays the different device models within that category

8. User-Created Packet box - Users can create highly-customized packets to test their topology from this area, and the results are displayed as list.

d) Analyse the behaviour of network devices using CISCO PACKET TRACER Simulator.

1. from the network component box, click and drag-and-drop the below components:

   a. 4 Generic PCs and One HUB
   b. 4 Generic PCs and One Switch

2. Click on Connections;

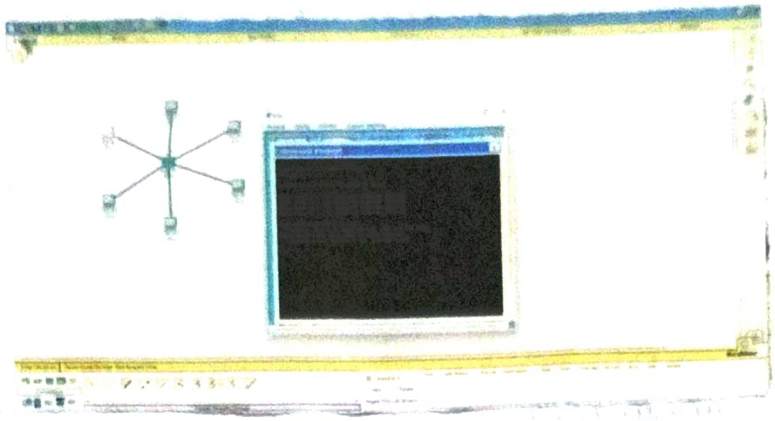
   a. click on copper Straight-Through Cable.
   b. Select one of the PC and connect it to HUB using the cable. the Link LED should glow on green, indicating that the link is up. similarly connect remaining 3 PCs

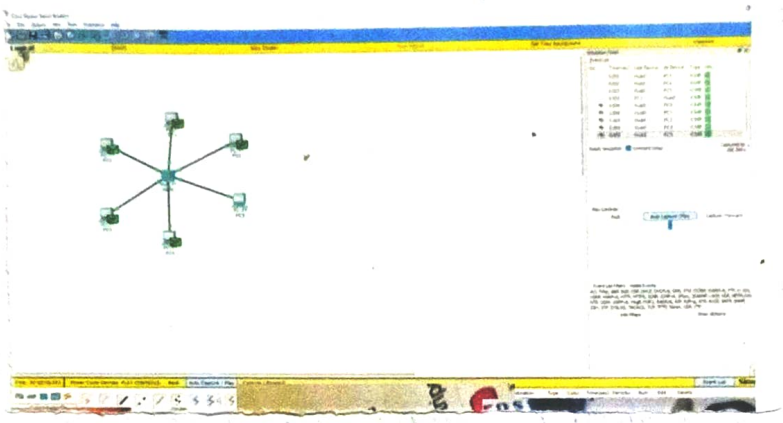
3. Click on the PCs connected to hub, go to the Desktop tab, click on IP Configuration and enter an IP address and Sub network mask.

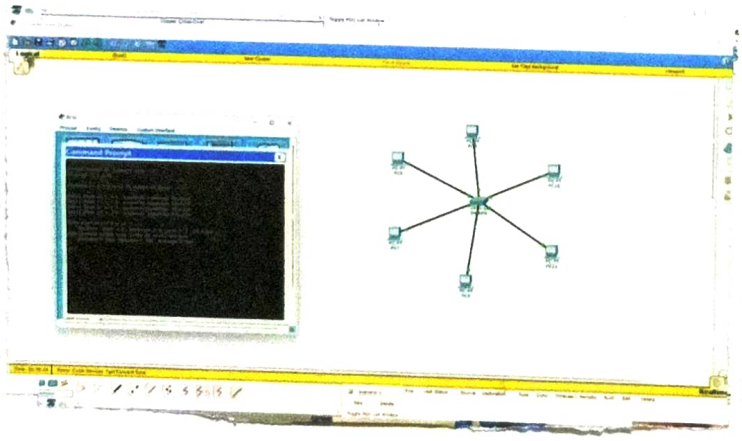
4. Observer the flow of PDU from Source PC to destination PC by selecting the Realtime mode of Simulation

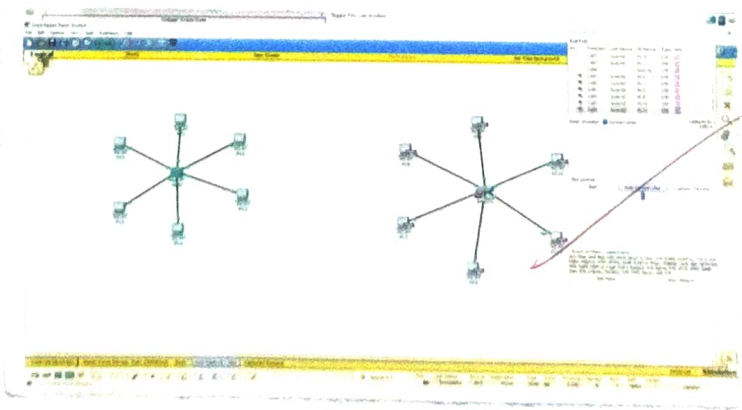
5. Repeat Step #3 to Step #5 for the PCs connected to the Services

6. Observe how HUB and Switch are forwarding the PDU and write your observation and conclusion about the behaviours of Switch and HUB



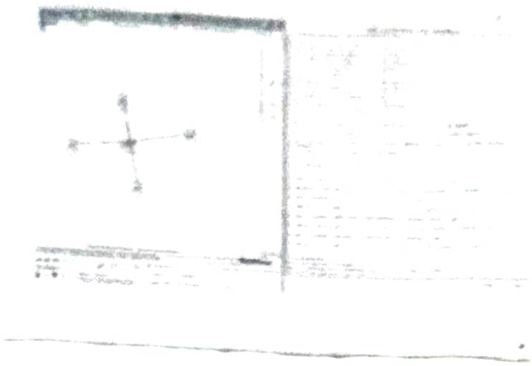


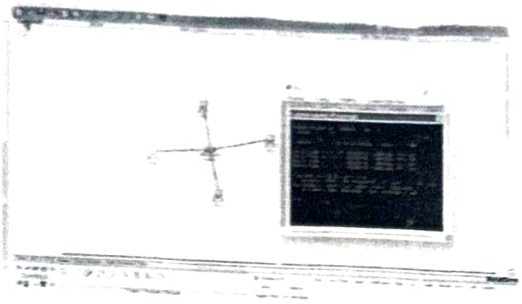

Command Prompt

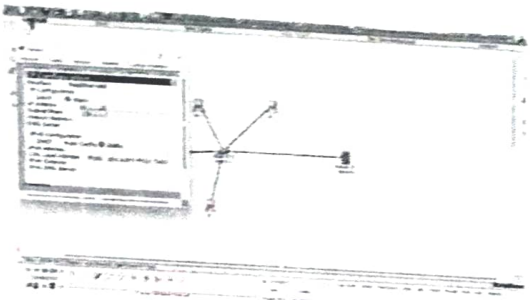


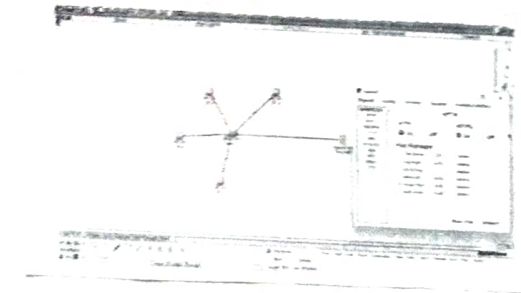
Result :-

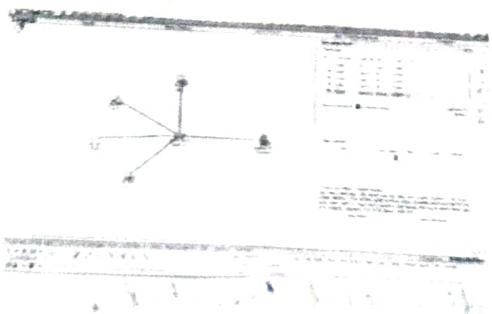
The Packet tracker tool Installation and User Interface overview is studied and Executed.

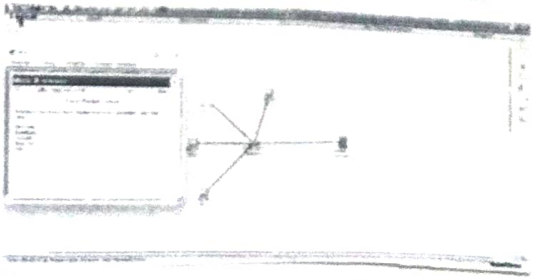












Experiment-4

Setting Up a Local Area Network (LAN)

Aim:- To design, set up, and configure a Local Area Network (LAN) connecting four Computers through a network Switch, assign Static IP addresses to all devices, Verify Connectivity Using the Ping Command, and demonstrate resource / data Sharing across the network.

Equipment Required

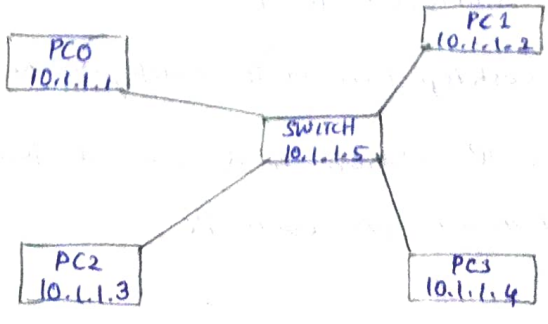
S.NO	Equipment	Quantity	Specification
1	Personal Computers (PC)	4	Ethernet (NIC) enabled
2	Network Switch	1	8/16/24 port, unmanaged/managed
3	Ethernet Cables (cat5e / cat6)	4	Straight-through, RJ-45 Connectors.

Theory

A Local Area Network (LAN) is a network that Connects Computers and devices within a limited geographical area such as a home, office, or laboratory, allowing them to share resources such as files, printers, and Internet connections

A Star Topology is used in this experiment, where all end devices (PCs) are individually connected to a central network Switch. The switch operates at Layer 2 of the OSI model and forwards data frames b/w connected devices based on their MAC addresses.

Network diagram Star topology (4 PCs + Switch) LAN



5. IP Address Configuration Table

Device	IP Address	Subnet Mask	Connected To
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch (Management)	10.1.1.5	255.0.0.0	VLAN 1 Interface

6. Implementation in Cisco Packet Tracker

Since Physical hardware was limited, the LAN was also implemented and verified using Cisco Packet Tracer Simulation Software, following the step below.

Step 1:- Place the Devices

1. Open Cisco packet Tracer
2. From the device category panel, select End Devices and drag 4 PCs onto the Workspace.
3. Select Switches and drag 1 Switch (eg 2960 series) onto the workspace

Step 2:- Cable the Devices

4. Click Connections → select ~~End Device~~ Copper straight Through Cable
5. Click each PC, then click the Switch, and select an available FastEthernet Port to complete each connection.
6. Repeat for all four PCs (Port Fa0/1 to Fa0/4). Green link lights confirm a successful Physical connection

Step 3:- Assign IP Addresses to PCs

7. Click a PC → Desktop tab → IP Configuration
8. Enter the static IP Address and Subnet Mask as per the IP table (Section 5) for each PC.

Step 4 : Configure the Switch Management IP (Optional)

9. Click the Switch → CLI tab
10. Enter the following commands to assign a management IP.

```
enable
Configure terminal
interface Vlan 1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit
```

Step 5:- Verify Connectivity Using ping

11. Click a PC → Desktop tab → Command prompt
12. Use the ping command
13. Succesful "Reply from ...." menage confirm Connectivity

Step 6:- Demonstrate Data Sharing (Server Simulation)

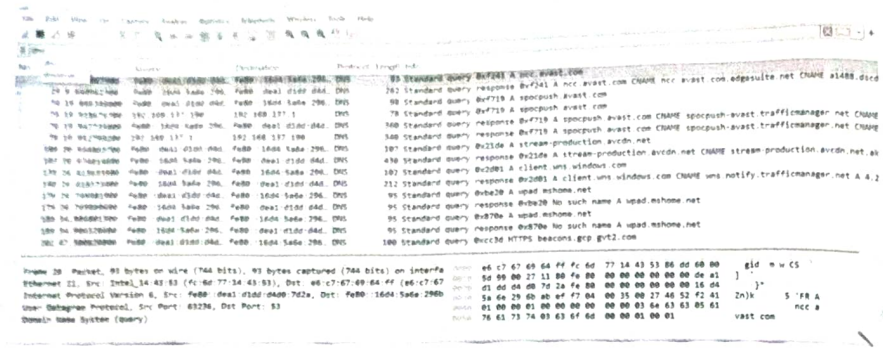
14. Add a Server device (End devices → Server-PT) and connect it to the switch.
15. Assign it an IP Address (eg 10.1.1.10, Subnet mask 255.0.0.0) via its desktop → IP Configuration tab.
16. Open the Server's Services (Config) tab and enable the HTTP service.
17. From the another PC, Open the Web Browser (Desktop tab) and enter the Server's IP address to load the page, Confirming Successful data transfer across the LAN.

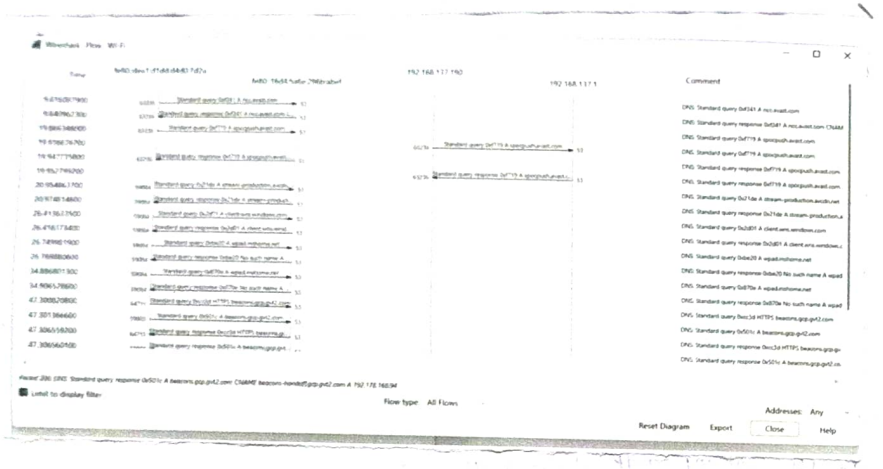
Step 7:- Observe Packet Flow in Simulation Mode

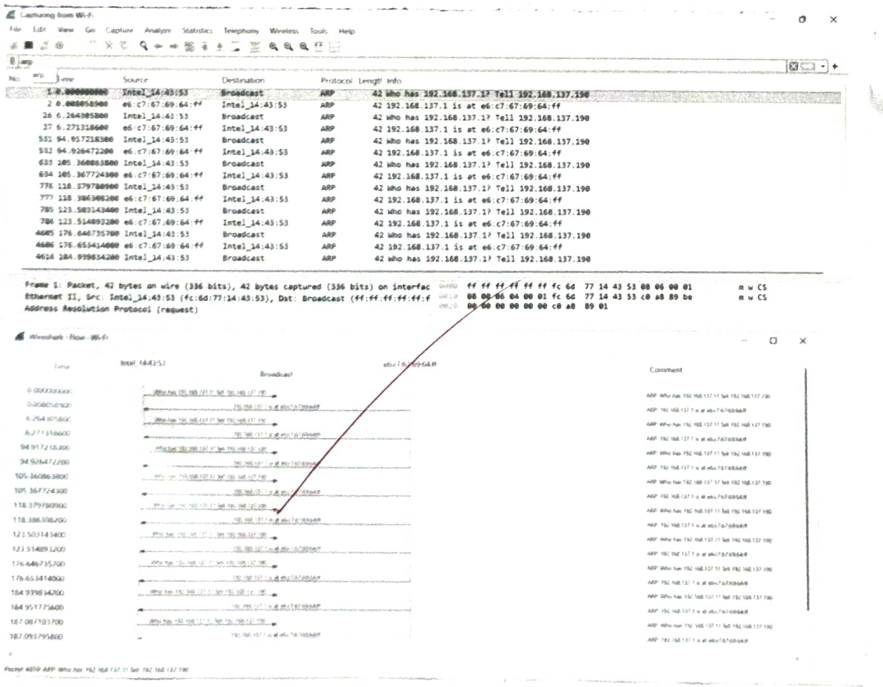
18. Switch at Ping to simulation mode (bottom-right of Workspace)
19. Send a Ping or web request b/w 2 devices and use the Play/Capture controls to step through the Packet

Observation :-

All four PCs were successfully connected to the Switch in a star topology and assigned unique static IP address within the same subnet (10.0.0.0/255.0.0.0), Ping tests b/w all the devices returned Successful replies.







Experiment-5

18/8/26

Experiments on packet Capture tool : Wireshark

Aim :- Experiment and Study on packet Capture tool : Wireshark

Packet Sniffer

- Sniff messages being sent / received from / by your Computer
- Store and display the content of the various protocol fields in the message.
- passive program
  - never sends packet itself
  - no packets addressed to it.
  - receives a copy of all packets (sent / received)

Packet Sniffer structure Diagnistic Tool

Tcpdump

Eg. tcpdump -enx host 10.129.41.2 -w exe3.out

Wireshark

Wireshark -r exe3.out

Desciption

Wireshark

Wireshark, a network analysis tool formely known as Ethereal Captures packets in real time and display them in human-readable format.

Wireshark includes filters, color coding and other features that let you dig deep into network traffic and individual packets.

Capturing and analysis Packets using Wireshark Tool

TO filter, capture, view, packets in Wireshark Tool

Capture 100 packets from the Ethernet IEEE 802.3 LAN Interface and save it.

Procedure :-

* Select Local Area Connection in Wireshark
* Go to Capture option
* Select stop capture automatically after 100 packets
* Then click Start capture
* Save the packets

2. Create a Filter to display only TCP/UDP packets, inspect the packets and provide the flow graph.

procedure :-

* Select Local Area Communication in Wireshark.
* Go to Capture option
* Select Stop Capture automatically after 100 packets
* Then click Start Capture
* Search TCP Packets in Search bar.
* To See flow graph click Statistics Flow graph
* Save the packet

3. Create Filter to display only ARP packets and inspect the packets

procedure :-

* Go to Capture Option
* Select top capture automatically after 100 packets
* Then click Start capture.
* Search ARP packets in Search bar
* Save the packet.

3. Create a filter to display only DNS packet and provide the flow graph

procedure :-

* Go to capture option
* Select the Stop capture automatically after 100 packets
* Then click Start capture
* Search DNS packets in search bar.
* To use flow graph click statistics flow graph
* Save the packet.

4. Create a filter to display only HTTP packet & inspect the packet.

procedure :-

Select Local Area connection in Wireless
Go to capture option
Select the Stop capture automatically after 100 packets
Then click Start capture
Search HTTP Packets in Search

5. Create a filter to display only IP / ICMP packet and inspect with packet

procedure

Select local Area connection in wireshark
Go to capture option
Select Stop capture automatically after 100 packets
Then click Start capture
Search ICMP / IP packet in search bar
Save the packets.
Create a filter to display only DHCP packets & inspect the packets.

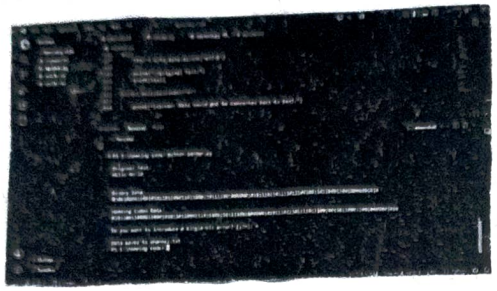
6. Create a filter to display only DHCP packets & inspect the packets

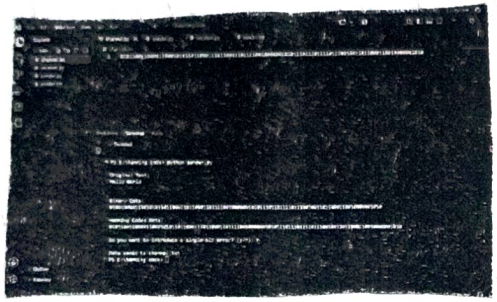
procedure :-

Select Local Area network connection in wireshark
Go to capture option
Select opto capture automatically after 100 packets
Then click Start capture
Search DHCP packets in Search box
Save the packet

Result :-

Thus, Studied the Packet capture tool : wireshark successfully









17/8/24

EXP-6

HAMMING CODE

Aim :- Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

Error Correction at Data Link layer

Hamming code is set of error-correction codes that can be used to detect and correct the errors that can be occur when the data

Create sender program with below feature.

Input to sender file should be a text of any length.
program should convert the text
Apply hamming code concept on the binary data and add redundant bits to it.
Save this output in a file called channel
Create a receiver program with below feature.
Receiver prg should read the input from channel file
Apply hamming code on the binary data check for errors
If there is an error, display the position of the error.
Else remove the redundant bits and convert the binary data to ascii and display the output

Write the code

Sender.py

```
def find_parity_bits(data_length):
    r = 0
    while (2**r) < (data_length + r + 1):
        r += 1
    return r

def hamming_encode(data):
    data_length = len(data)
    parity_bits = find_parity_bits(data_length)
    total_length = data_length + parity_bits
    hamming = ['0'] * (total_length + 1)
    data_index = 0
    for position in range(1, total_length + 1):
        if position & (position - 1):
            hamming[position] = data[data_index]
            data_index += 1
    for i in range(parity_bits):
        parity_position = 2**i
        parity = 0
        for position in range(1, total_length + 1):
            if position & parity_position:
                parity ^= int(hamming[position])
        hamming[parity_position] = str(parity)
    return ''.join(hamming[1:])

with open("sender.txt", "r", encoding="utf-8") as file:
    text = file.read()
print("\n original Text :")
print(text)
binary_data = ''.join(
    format(byte, '08b')
    for byte in text.encode('utf-8')
)
print("\n binary Data :")
print(binary_data)
encoded_data = hamming_encode(binary_data)
print("\n Hamming Coded Data :")
print(encoded_data)
```

```
choice = input(
    "\n Do you want to introduce a single-bit error? (y/n): "
).lower()
channel_data = encoded_data
if choice == 'y':
    error_position = 40
    if error_position <= len(channel_data):
        bit_list = list(channel_data)
        original_bit = bit_list[error_position - 1]
        if original_bit == '0':
            bit_list[error_position - 1] = '1'
        else:
            bit_list[error_position - 1] = '0'
        channel_data = ''.join(bit_list)
        print("\n Single-bit error introduced!")
        print("Error position in channel:", error_position)
    else:
        print("\n Error position is larger than the data")
        print("No error introduced.")
with open("channel.txt", "w") as file:
    file.write(channel_data)
print("\n Data saved to channel.txt")
```

Sender.txt

Hello World

receiver.py

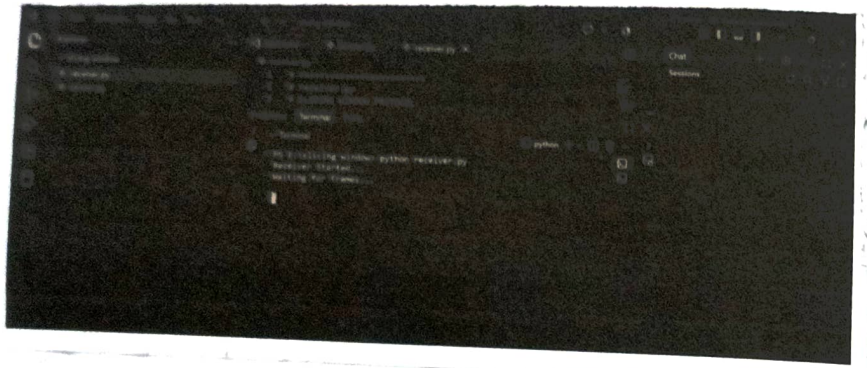
```
def hamming_decode(received):
    n = len(received)
    hamming = ['0'] + list(received)
    parity_bits = 0
    while (2 ** parity_bits) <= n:
        parity_bits += 1
    error_position = 0
    for i in range(parity_bits):
        parity_position = 2**i
        parity = 0
        for position in range(1, n+1):
            if position & parity_position:
                parity ^= int(hamming[position])
        if parity != 0:
            error_position += parity_position
```

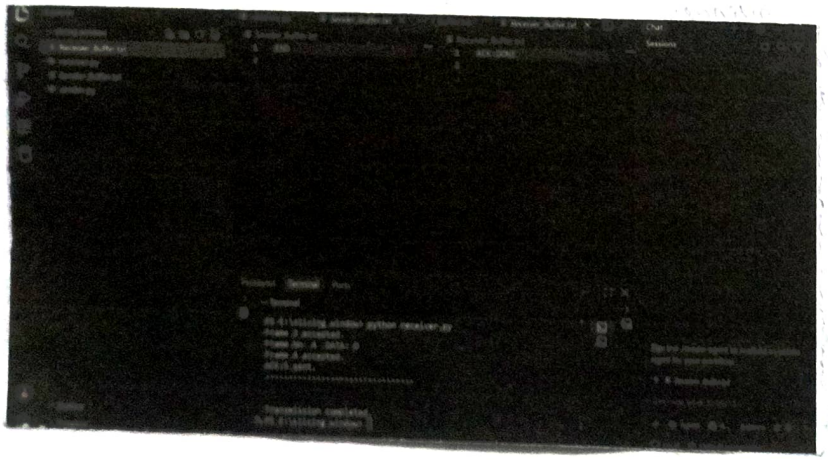
```
While start < len(frames):
    end = min(start + window_size, len(frames))
    current_window = frames[start : end]
with open(SENDER_BUFFER, "w", encoding = "utf-8") as file:
    for frame in current_window:
        file.write(
            f"FRAME_NO:{frame['no']},DATA:{frame['data']}\n"
        )
    print("Frames sent:")
    for frame in current_window:
        print(
            f" FRAME No.: {frame['no']}, DATA: {frame['data']}")
    Printf(" Sender_Buffer.txt updated.")
    time.sleep(5)
    if not os.path.exists(RECEIVER_BUFFER):
        Print("Receiver_Buffer.txt not found.")
        print("Please run receiver.py")
        time.sleep(2)
        continue
with open(RECEIVER_BUFFER, "r", encoding = "utf-8") as file:
    response = file.read().strip()
    Print("\n Receiver Response:", response)
if response.startswith("Ack:"):
    try:
        ack_no = int(response.split(":")[1].strip())
    except ValueError:
        print("Invalid Ack received.")
        continue
    Print("Acknowledgement Number:", ack_no)
    if ack_no == end:
        Print("Ack is correct")
        Print("Sending next window...\n")
        start = ack_no
    else
        print("Unexcepted Ack.")
        print("Reseeding current window...\n")
else
    print("Invalid Reciver response")
with open(SENDER_BUFFER, "w", encoding = "utf-8") as file
    file.write("END\n")
print("All frames transmitted successfully. Transmission
    completed.")
```

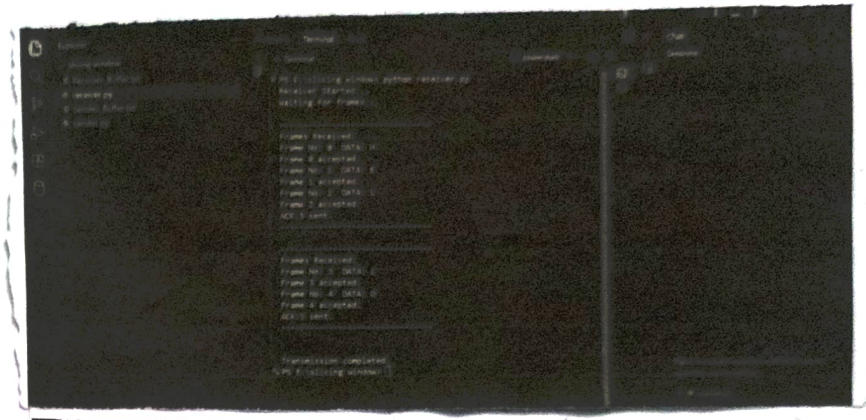
receiver.py

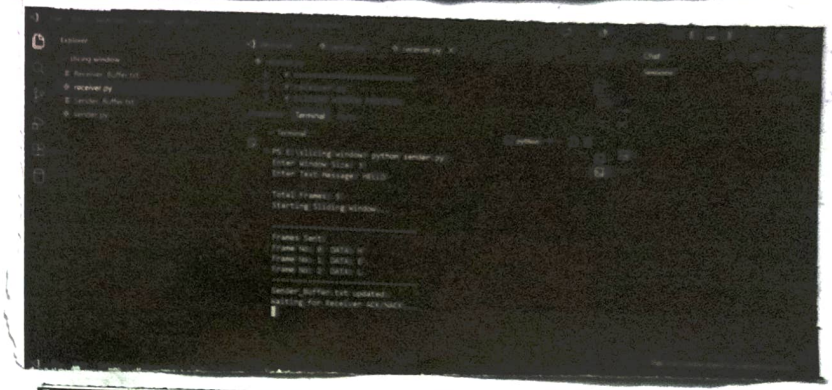
```
import time
import os
SENDER_BUFFER = "Sender_Buffer.txt"
RECEIVER_BUFFER = "Receiver_Buffer.txt"
excepted = 0
last = ""
print("Receiver started....")
Print("waiting for frames...\n")
while True:
    if not os.path.exists(SENDER_BUFFER):
        time.sleep(1)
        continue
with open(SENDER_BUFFER, "r", encoding = "utf-8") as f
    content = f.read()
    if content == last:
        time.sleep(1)
        continue
    last = content
    lines = content.strip().splitlines()
if lines and lines[0] == "END":
    with open(RECEIVER_BUFFER, "w") as f
        f.write("ACK:DONE")
        print("Transmission completed.")
        break
print("\nFrame Received:")
error = False
for line in lines
    try:
        parts = line.split(",")
        no = int(parts[0].split(":")[1])
        data = parts[1].split(":",1)[1]
    except:
        error = True
        break
```

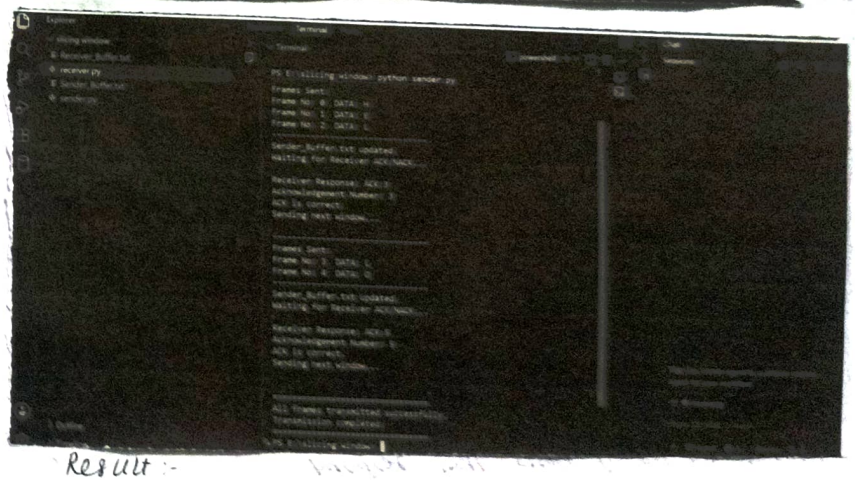
```
print(f"FRAME NO: {no} DATA: {data}")
if no == expected:
    print(f"FRAME {no} accepted")
    expected += 1
else:
    print(f"ERROR: Expected Frame {expected}")
    error = TRUE
    break
```











Result :-

The sliding window protocol was successfully implemented using Python.

Verified.

28/9/25

EXP-8

N-map Live Host Discovery

Aim: To discover live hosts on a network using N-map host discovery scans - ARP scan, ICMP scan, and TCP/UDP ping scan - and to understand which protocol operates at which layer of the TCP/IP model, and when each one applies.

INTRODUCTION:

Nmap ("Network Mapper") is a free, open-source network scanning tool created by Gordon Lyon.

Nmap can determine whether a host is alive using four different protocol, one at each relevant layer of TCP/IP

- ARP (Link layer) - for host on same local subnet
- ICMP (Network layer) - Echo, timestamp
- TCP (Transport layer) - SYN and ACK ping probes
- UDP (Transport layer) - relies on an ICMP "port unreachable"

Procedure

1. Log in to TryHackMe and open the N-map Live Host Discovery room
2. Complete Task 1-3, covering Introduction, Subnetworking and TCP/IP layer concepts, and answer the simulator-based question
3. Deploy the lab network in Task 4 and start the AttackBox. Note the Target subnet / CONNECTION_IP
4. perform an ARP scan using
   sudo nmap -PR -sn <subnet>/24
   Record the no of hosts that respond
5. perform an ICMP Echo scan using
   sudo nmap -PE -sn <subnet>/24
   Compare the result with the ARP scan
6. perform the following scans:
   - TCP SYN ping (-PS)
   - UDP ping (-PU)
   - TCP ACK ping (-PA) using sudo

compare their results and note the required privilege level.

8. Optionally Capture the network traffic using tcpdump or wireshark on the tun0 interface to observe the ARP, ICMP, TCP and UDP packets
9. run the following commands
   nmap -sL <subnet>/24
   nmap -R <subnet>/24
10. Complete the knowledge-check and Nmap basic port scans room.



Result :-

The N-map live host Discovery lab was successfully completed.

22/9/24

EXP-9

SUBNETTING in CISCO PACKET TRACER Simulator

Aim: Implementation of subnetting in cisco packet tracer simulator

Classless IP subnetting is a technique that allows for more efficient use of IP addresses by allowing for subnet masks that are not just the default masks for each IP class

CREATING A NETWORK TOPOLOGY

To create a network topology in packet Tracer select the "new" button in top left corner, then select "network" and "Generic".

ADDING THE DEVICES

To add a device, select the device from the bottom left corner and drag it onto the network topology.

SUBNETTING

To subnet the network address of 192.168.1.0/24 to provide enough space for a least 5 addresses for end devices, the switch and the router.

The IP addressing for the network shown in the topology can be as follows.

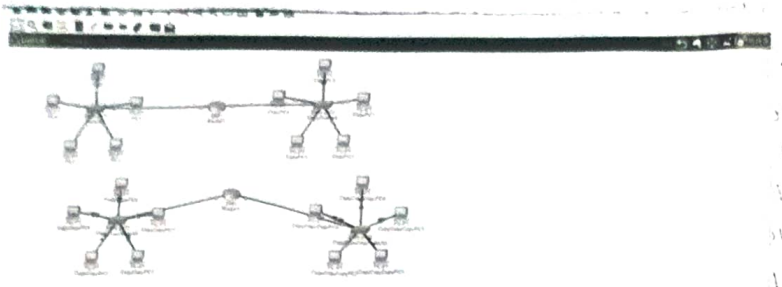
Router R1
GigabitEthernet0/0 : 192.168.1.1
GigabitEthernet0/1 : 192.168.2.1
Switch S1:
FastEthernet 0/1 : 192.168.1.0/27
PC1 : 192.168.1.11
PC2 : 192.165.1.12
PC3 : 192.168.1.13
PC4 : 192.168.1.14
PC5 : 192.168.1.15
FastEthernet 0/2 : 192.168.2.0/27
PC1 : 192.168.2.11
PC2 : 192.168.2.12
PC3 : 192.168.2.13
PC4 : 192.168.2.14
PC5 : 192.168.2.15

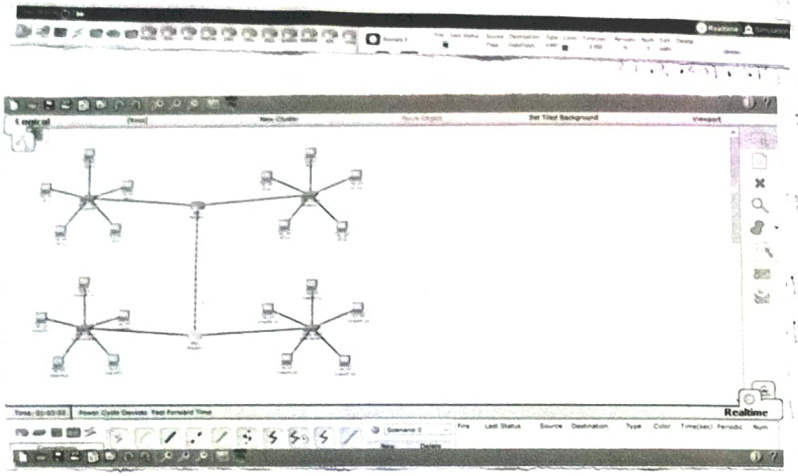
Router R2:
FastEthernet 0/0 : 192.168.3.1
FastEthernet 0/1 : 192.168.4.1
Switch S2
FastEthernet 0/1 : 192.168.3.0/27
PC1 : 192.168.3.11
PC2 : 192.168.3.12
PC3 : 192.168.3.13
PC4 : 192.168.3.14
PC5 : 192.168.3.15
FastEthernet 0/2 : 192.168.4.0/27
PC1 : 192.168.4.11
PC2 : 192.168.4.12
PC3 : 192.168.4.13
PC4 : 192.168.4.14
PC5 : 192.168.4.15

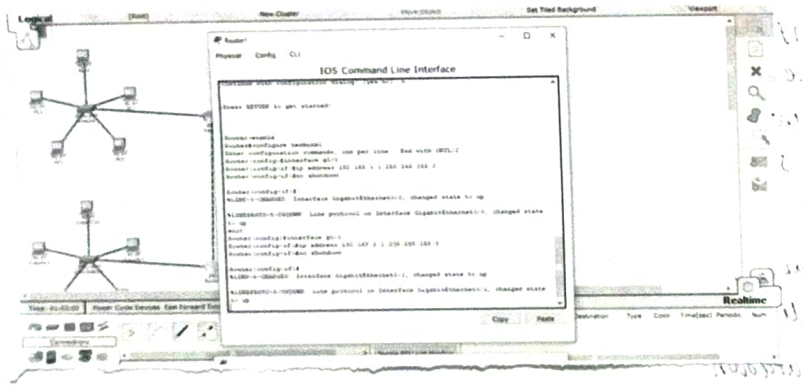
CONFIGURING THE DEVICE

enable
configure terminal
interface FastEthernet 0/0
ip address {IP address} {subnet mask}
no shutdown
exit

interface FastEthernet 0/1
ip address {IP address} {subnet mask}
no shutdown
exit







Result :-

The network was successfully designed and configured in Cisco Packet Tracer using classless IP subnetting.

Date :- 05/10/26

EXP-10

Internetworking with routers in CISCO PACKET TRACER

Aim :- Internetworking with routers in CISCO packet Tracer simulator

Design and Configure a simple internetwork using a router

In this network, a router and 2 PCs are used. Computers are connected with routers using a copper straight-through Cable. After forming the network, to check network connectivity a simple PDU is transferred from PC0 to PC1.

procedure :-

1. Open Cisco packet Tracer and place Router 1, PC0, and PC1 and connect straight copper cables.
2. Select Router and select CLI and press Enter
3. Enter the following commands.

```
Router > enable
Router# configure terminal
Router (config)# interface FastEthernet0/0
Router (config-if)# ip address 192.168.10.1 255.255.255.0
Router (config-if)# no shutdown
```

4. Configure the second interface :

```
Router (config-if)# interface FastEthernet0/1
Router (config-if)# ip address 192.168.20.1 255.255.255.0
Router (config-if)# no shutdown
```

5. Configure PC0

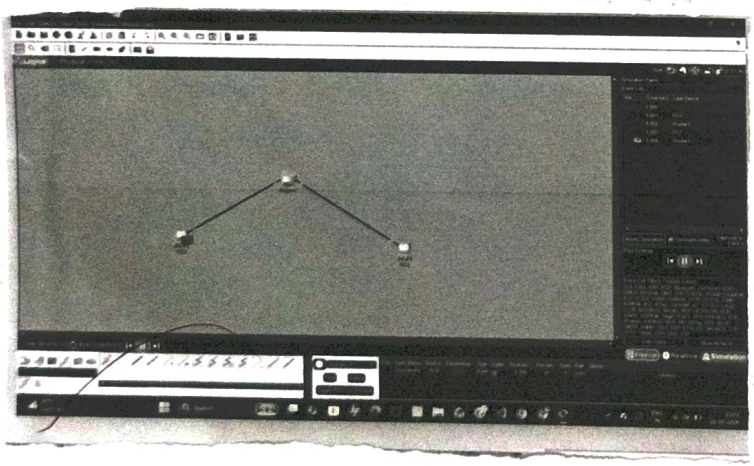
Go to PC0 - Desktop → IP Configuration

IP address : 192.168.10.2

Subnet Mask : 255.255.255.0

Default Gateway : 192.168.10.1

6. Configure PC1
   Go to PC1 - Desktop → IP configuration
   IP Address : 192.168.20.2
   Subnet Mask : 255.255.255.0
   Default Gateway : 192.168.20.1
7. Connect PC0 FastEthernet0 to Router1 FastEthernet0/0 using a Copper Straight Through Cable
   Connect PC1 FastEthernet0 to Router1 FastEthernet0/1 using a copper straight-Through Cable.
9. Wait for the interface to turn green/up.
10. Test the connection by opening PC0 → Command prompt and entering
    ping 192.168.20.2
11. If replies are received, the network configuration is successful.

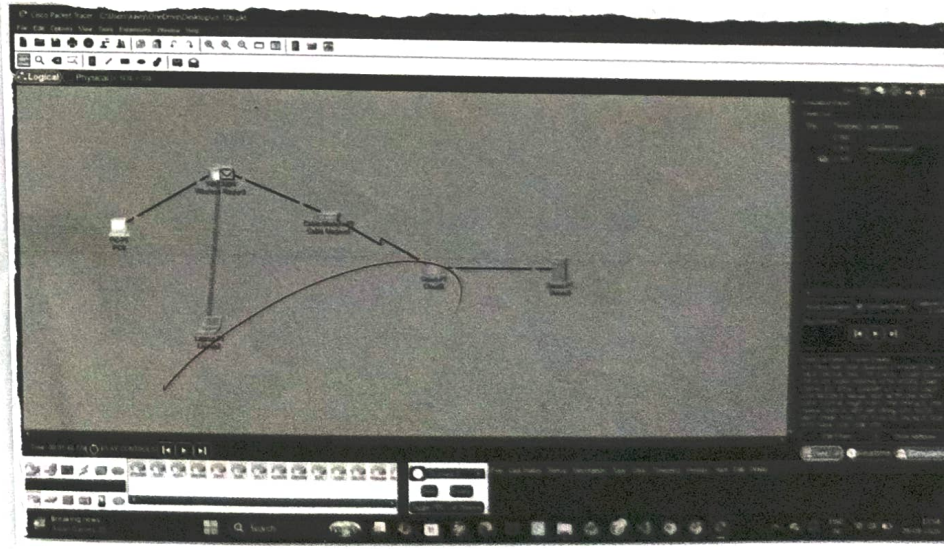


10 b) Wireless Router, DHCP server

Aim :- To design and configure an internetworking using a wireless router, DHCP server & internet cloud

Observation :-

1) A wireless router, PC, laptop, cable modem, internet cloud of cisco.com server are connected
2) The wireless router is configured with the SSID Homenetwork.
3) DHCP is enabled on the wireless router
4) The PC obtains its IPV4 address automatically through DHCP
5) The cisco.com server is configured as a DHCP & DNS server.
6) The PC receives an address in the 192.168.0.x range
7) Connectivity is Verified using ping cisco.com



Date:
6/10/24

11. Simulate RIP using cisco packet Tracer.

Aim:- The implement the simulation of RIP using cisco packet Tracer

Observations

1. Assign ip address to PCs

ip addresses, Subnet masks, and default gateways were assigned to all PCs using Desktop → IP configuration in cisco packet Tracer.

2. Router0 Configuration

```
Router> enable
Router# configure terminal
Router(config)# interface fastEthernet 0/0
Router(config-if)# ip address 10.0.0.1 255.0.0.0
Router(config-if)# no-shutdown
Router(config-if)# exit

Router(config)# interface serial 0/0/0
Router(config-if)# ip address 192.168.1.249 255.255.255.252
Router(config-if)# clock rate 64000
Router(config-if)# bandwidth 64
Router(config-if)# no shutdown
Router(config-if)# exit.
```

3 Router1 Configuration

```
Router(config)# interface serial 0/0/1
Router# (Configuring) terminal # ip address 192.168.1.254
255.255.255.252
Router(config-if)# clock rate 64000
Router(config-if)# bandwidth 64
Router(config-if)# no shutdown
Router(config-if)# exit.
```

Router1 Configuration

```
Router> enable
Router# Configure terminal
Router(config)# interface serial 0/0/0
Router(config-if)# ip address 192.168.1.250 255.255.255.252
Router(config-if)# no shutdown
Router(config-if)# exit

Router(config)# interface serial 0/0/1
Router(config-if)# ip-address 192.168.1.246 255.255.255.252
Router(config-if)# clock rate 64000
Router(config-if)# bandwidth 64
Router(config-if)# no shutdown
Router(config-if)# exit
```

Router2 Configuration

```
Router>enable
Router# configure terminal
Router(config)# interface fastEthernet 0/0
Router(config-if)# ip address 20.0.0.1 255.0.0.0
Router(config-if)# no shutdown
Router(config-if)# exit

Router(config)# interface serial 0/0/0
Router(config-if)# ip address 192.168.1.245 255.255.255.252
Router(config-if)# no shutdown
Router(config-if)# exit
```

5. Configure RIP protocol

Router 0:

Router(config)# router rip

Router(config-router)# network 10.0.0.0

Router(config-router)# network 192.168.1.252

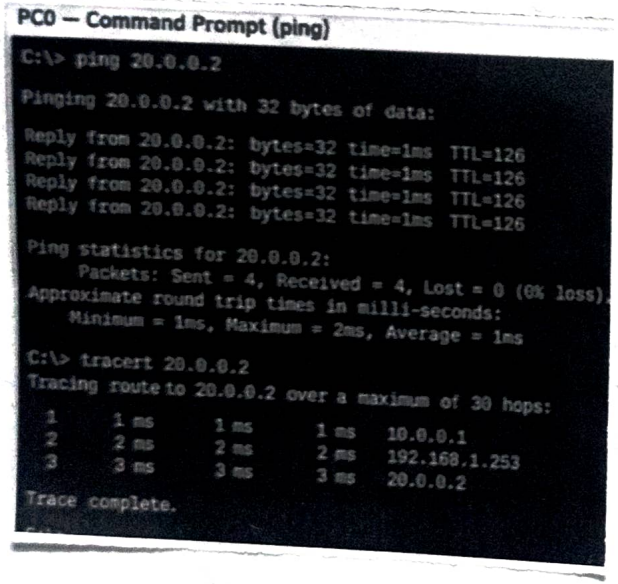
Router 1:

Router(config)# router rip

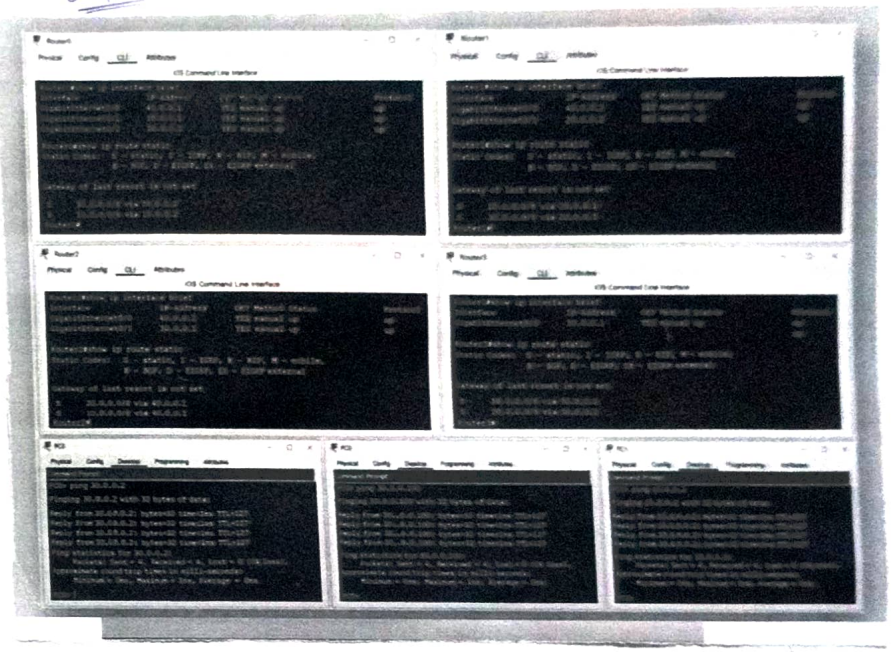
Router(config-router)# network 192.168.1.244

Router(config-router)# network 192.168.1.248

OUTPUT :-



OUTPUT :-



Result :-

IP addressing and router interface were configured

10/10/12 12. End-End communication at Transport Layer

a) Implement echo client server using TCP/UDP sockets
b) Implement a chat program using socket Programming

TCP

```
import socket
import threading

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen(1)
    print("[SERVER] Listening on 127.0.0.1:5000")
    conn, addr = server.accept()
    print(f"[SERVER] Connected by {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit':
            print("[SERVER] connection closed by client.")
            break
        print(f"[SERVER] Received: {data}")
        conn.send(f"server received: {data}".encode())
    conn.close()
    server.close()
def start_client():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] Connected to server")
    while True:
        msg = input("Enter message (or 'quit' to exit): ")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}")
    client.close()
if __name__ == "__main__":
    server_thread = threading.Thread(target = start_server, daemon = True)
    server_thread.start()
    start_client()
```

UDP

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    while True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] client {addr} ended the connection.")
            break
        print(f"[SERVER] Recieved from {addr}: {msg}")
        s.sendto(data, addr)
    s.close()
    print("[SERVER] stopped.")
def udp_client():
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter the message (or 'quit' to stop): ")
        c.sendto(msg.encode(), server)
        if msg.lower() == 'quit':
            print("[CLIENT] closing connection....") break
        data, _ = c.recvfrom(1024)
        print("Echo from server:", data.decode())
    c.close()
    print("[CLIENT] stopped.")
if __name__ == "__main__":
    threading.Thread(target = udp_server, daemon = True).start()
    udp_client()
```

OUTPUT :-

```
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 53458)Enter message (or 'quit' to exit):
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
Enter message (or 'quit' to exit):
========== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 60328)Enter message (or 'quit' to exit):
HI
[SERVER] Received: HI
[SERVER RESPONSE] Server received: HI
Enter message (or 'quit' to exit): quit
[SERVER] Connection closed by client.

Enter message (or 'quit' to stop): [SERVER] Running on 127.0.0.1:12000hi

[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from Server: hi
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 49670): hello
Echo from Server: hello
Enter message (or 'quit' to stop): how are you
[SERVER] Received from ('127.0.0.1', 49670): how are you
Echo from Server: how are you
Enter message (or 'quit' to stop): quit
[CLIENT] Closing connection...[SERVER] Client ('127.0.0.1', 49670) ended the connection.

[CLIENT] Stopped.[SERVER] Stopped.
```

Result :-

Thus echo client server using TCP, UDP socket, Chat program using socket programming implement.

18/10/24

13 Implement you own ping prg

```
import os
import platform
import subprocess
def ping(host="google.com", count=4):
param="-n" if platform.system().lower()=="windows" else "-c"
command=["ping", param, str(count), host]
try:
output=subprocess.check_output(command, universal_newlines=True)
print(output)
except Exception as e:
print(f"ping failed: {e}")
if __name__=="__main__":
ping("8.8.8.8")
```

OUTPUT :-

```
C:\Users\kaushika\OneDrive\Desktop>python exp13.py

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=10ms TTL=118
Reply from 8.8.8.8: bytes=32 time=11ms TTL=118
Reply from 8.8.8.8: bytes=32 time=10ms TTL=118
Reply from 8.8.8.8: bytes=32 time=12ms TTL=118

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 10ms, Maximum = 12ms, Average = 10ms
```

Result :-

Thus, own ping program is implemented.

13/10/24 14. Write a code using RAW sockets to implement packet sniffing

```
from Scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP

def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst

    protocol_name = ""
    if protocol == 1:
        protocol_name = "ICMP"
    elif protocol == 6:
        protocol_name = "TCP"
    elif protocol == 17:
        protocol_name = "UDP"
    else:
        protocol_name = "Unknown Protocol"

    print(f"Protocol: {protocol_name}")
    print(f"Source IP: {src_ip}")
    print(f"Destination IP: {dst_ip}")
    print("-"*50)
sniff(iface='wi-fi', prn=packet_callback, filter="ip",
                                        store=0)
```

OUTPUT :-

```
C:\Users\kaushika\OneDrive\Desktop>python exp14.py
Protocol: TCP
Source IP: 104.16.103.112
Destination IP: 192.168.1.4
--------------------------------------------------
Protocol: TCP
Source IP: 192.168.1.4
Destination IP: 104.16.103.112
--------------------------------------------------
Protocol: TCP
Source IP: 104.16.103.112
Destination IP: 192.168.1.4
--------------------------------------------------
Protocol: TCP
Source IP: 192.168.1.4
Destination IP: 172.188.155.25
--------------------------------------------------
Protocol: TCP
Source IP: 172.188.155.25
Destination IP: 192.168.1.4
--------------------------------------------------
Protocol: TCP
Source IP: 192.168.1.4
Destination IP: 172.188.155.25
--------------------------------------------------
```

Result :-

Thus, the program is written using RAW sockets to implement packet sniffing

Completed
13/10/24